Security Audit Report

Neutron Auction and Undelegations Contract, Chain Upgrade

Phase 2

Report 2

v1.0

April 9, 2026

Table of Contents

Table of Contents 2
License 4
Disclaimer 5
Introduction 6
- Purpose of This Report 6
- Codebase Submitted for the Audit 6
- Methodology 8
- Functionality Overview 8

How to Read This Report 9
Code Quality Criteria 10
Summary of Findings 11
Detailed Findings 13
- 1. Misconfigured governance module permissions causing consensus-halting panic on deposit burn 13
- 2. Any CosmWasm contract can add cron schedules, bypassing authorization 13
- 3. ValidatorDstAddress may equal ValidatorSrcAddress in redelegation messages, causing upgrade failure 14
- 4. calcRedelegations uses delegation shares as token amounts, producing incorrect results when validators have been slashed 14
- 5. nil DistributionKeeper passed to the wasm keeper prevents distribution messages in CosmWasm contracts 15
- 6. Incorrect BeginBlocker ordering leading to inconsistent reward accounting 16
- 7. Identical minDeposit and expeditedMinDeposit remove economic distinctions between normal and expedited proposals 16
- 8. Incomplete fund migration leading to permanent token lock in deprecated module accounts 17
- 9. 100% minInitialDepositRatio creates high entry barriers and renders deposit periods meaningless 17
- 10. Zero proposalCancelRatio enables costless governance spam 18
- 11. PuppeteerAdmin retains migration permission post-upgrade 19
- 12. Truncated redelegation amounts diverge from internal accounting, leaving new validators underfunded 19
- 13. Zero-amount redelegation messages from TruncateInt halt the upgrade 20
- 14. setDefaultParams name misrepresents scope 20
- 15. BurnFunds error prints module name instead of address 21
- 16. removeSchedule error references removed admin module 21
- 17. Stale comment incorrectly states that GovMsg::VoteWeighted does not work with Neutron 21
- 18. Multiple hardcoded values in the upgrade handler will cause deployment failure 22
- 19. SetupStaking writes staking params twice via two different code paths 23

20. BurnFunds fails hard if either account balance is zero at upgrade time 23
21. Transitive redelegation rule may block RedelegateDaoFunds if recent redelegations exist 24

License

THIS WORK IS LICENSED UNDER A CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE.

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by Hadron Labs to perform a security audit of Neutron Auction and Undelegations Contract, Chain Upgrade.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/neutron-org/neutron-private
Scope	The scope is restricted to the changes applied in the following pull requests: • https://github.com/neutron-org/neutron-private/pull/143 reviewed at commit `852898b855ca7cd2cbbcf97574ecade8102f5446`, base branch at `3ea79c95b272226bbbb17044a25584a359a3feb0`.
Fixes verified at commit	`fbc9fc4c6f4a34d1f2c1af1f72822f31728626c3`

	Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

The v11 upgrade migrates Neutron to the native Cosmos SDK governance module and updates the application's module set by removing x/revenue, x/feeburner, and adminmodule, while introducing gov, mint, and distribution.

As part of the upgrade process, the puppeteer contract is migrated to the auth-proxy implementation, DAO stake is redelegated to designated validators, and scheduled tasks are registered in the x/cron module to trigger tick and burn operations in the undelegations manager contract.

Funds held in the deprecated revenue module and the rewards contract are burned, ownership of the IBC Rate Limits contract is reassigned, and the state for the newly introduced x/distribution module is initialized manually due to its absence in prior versions.

Additionally, various module parameters are updated to reflect the new configuration.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	**Medium**	The upgrade strategy involves multiple moving parts.
Code readability and clarity	**Medium-High**	-
Level of documentation	**High**	The upgrade strategy is well described.
Test coverage	**Low-Medium**	Test coverage computed with `go test` is `41%`

Summary of Findings

No	Description	Severity	Status
1	Misconfigured governance module permissions causing consensus-halting panic on deposit burn	Critical	Resolved
2	Any CosmWasm contract can add cron schedules, bypassing authorization	Major	Resolved
3	`ValidatorDstAddress` may equal `ValidatorSrcAddress` in redelegation messages, causing upgrade failure	Major	Resolved
4	`calcRedelegations` uses delegation shares as token amounts, producing incorrect results when validators have been slashed	Major	Resolved
5	`nil DistributionKeeper` passed to the wasm keeper prevents distribution messages in CosmWasm contracts	Major	Resolved
6	Incorrect `BeginBlocker` ordering leading to inconsistent reward accounting	Major	Resolved
7	Identical `minDeposit` and `expeditedMinDeposit` remove economic distinctions between normal and expedited proposals	Major	Resolved
8	Incomplete fund migration leading to permanent token lock in deprecated module accounts	Minor	Resolved
9	100% `minInitialDepositRatio` creates high entry barriers and renders deposit periods meaningless	Minor	Acknowledged
10	Zero `proposalCancelRatio` enables costless governance spam	Minor	Acknowledged
11	`PuppeteerAdmin` retains migration permission post-upgrade	Minor	Resolved
12	Truncated redelegation amounts diverge from internal accounting, leaving new validators underfunded	Minor	Resolved
13	Zero-amount redelegation messages from TruncateInt halt the upgrade	Minor	Resolved

14	`setDefaultParams` name misrepresents scope	**Informational**	**Resolved**
15	`BurnFunds` error prints module name instead of address	**Informational**	**Resolved**
16	`removeSchedule` error references removed admin module	**Informational**	**Resolved**
17	Stale comment incorrectly states that `GovMsg::VoteWeighted` does not work with Neutron	**Informational**	**Resolved**
18	Multiple hardcoded values in the upgrade handler will cause deployment failure	**Informational**	**Acknowledged**
19	`SetupStaking` writes staking params twice via two different code paths	**Informational**	**Resolved**
20	`BurnFunds` fails hard if either account balance is zero at upgrade time	**Informational**	**Resolved**
21	Transitive redelegation rule may block `RedelegateDaoFunds` if recent redelegations exist	**Informational**	**Acknowledged**

Detailed Findings

1. Misconfigured governance module permissions causing consensus-halting panic on deposit burn

Severity: Critical

In `app/app.go:291-311`, the `maccPerms` map registers `govtypes.ModuleName` without the `authtypes.Burner` permission, resulting in a misconfigured governance module account lacking burn capabilities.

Despite this, the governance logic (based on Cosmos SDK v0.53.4) invokes deposit burning via `DeleteAndBurnDeposits`, which internally calls `bankKeeper.BurnCoins`. The `BurnCoins` implementation enforces permission checks and will trigger a panic if the calling module account does not have the Burner role.

Given that governance parameters (e.g., `BurnVoteVeto = true`) enable deposit burning under certain proposal outcomes (such as vetoed or canceled proposals), this creates a panic condition during the x/gov `EndBlocker` execution. A malicious actor can exploit this by submitting a proposal that triggers deposit burning (e.g., a vetoed proposal), causing all nodes to panic consistently and halting block production. The panic will occur on all nodes, effectively leading to a persistent chain halt until manual intervention.

Recommendation

We recommend aligning the governance module account permissions with the upstream Cosmos SDK by granting `authtypes.Burner` to `govtypes.ModuleName` in the `maccPerms` configuration.

Status: Resolved

2. Any CosmWasm contract can add cron schedules, bypassing authorization

Severity: Major

In `wasmbinding/message_plugin.go:811-815`, the `addSchedule` handler has had its authorization check replaced with `if false`, a dead branch that never executes. The handler then constructs a `MsgAddSchedule` using the governance module address as the hardcoded `Authority` field, which satisfies the cron keeper's authority check.

As a result, any CosmWasm contract can register arbitrary cron schedules that will execute at every block without any restriction on the caller.

An attacker-controlled contract can exploit this to spam cron schedules executing arbitrary messages at block begin or end, potentially griefing the chain, or causing unintended state changes.

Recommendation

We recommend replacing the dead `if false` branch with a real authorization check, for example verifying that `contractAddr` is on a governance-approved allowlist.

Note that `removeSchedule` in `wasmbinding/message_plugin.go:860` still restricts the caller to `params.Params.SecurityAddress`, meaning any contract can add a schedule but only the security DAO can remove one. The authorization model should be consistent across both entry points.

Status: Resolved

3. `ValidatorDstAddress` may equal `ValidatorSrcAddress` in redelegation messages, causing upgrade failure

Severity: Major

In `app/upgrades/nextupgrade/upgrades.go:542-544`, `calcRedelegations` constructs `MsgBeginRedelegate` messages using `delegation.ValidatorAddress` as `ValidatorSrcAddress` and `redelegations[newValIdx].ValidatorAddress` as `ValidatorDstAddress`.

If any validator in `NewValidatorSet` also appears in the current delegation set, the generated message may have an identical source and destination address. The Cosmos SDK rejects self-redelegations, and the resulting error would abort the upgrade handler.

Recommendation

We recommend adding a check in `calcRedelegations` to skip generating a redelegation message when `ValidatorSrcAddress == ValidatorDstAddress`.

Status: Resolved

4. `calcRedelegations` uses delegation shares as token amounts, producing incorrect results when validators have been slashed

Severity: Major

In `app/upgrades/nextupgrade/upgrades.go:516-518`, `totalDelegatedAmount` is computed by summing `delegation.Shares` across all delegations. In

`app/upgrades/nextupgrade/upgrades.go:544`, the resulting `take` value is passed directly as the coin amount in `MsgBeginRedelegate` via `take.TruncateInt()`.

However, `MsgBeginRedelegate.Amount` expects a token amount, not a share count. After a slash, a validator's exchange rate deviates from 1:1 (e.g., after a 50% slash, 100 shares are worth 50 tokens), so using shares as token amounts would produce incorrect redelegation sizes.

Recommendation

We recommend converting shares to token amounts using `validator.TokensFromShares(delegation.Shares).TruncateInt()` before summing and before constructing `MsgBeginRedelegate`.

Status: Resolved

5. `nil` `DistributionKeeper` passed to the wasm keeper prevents distribution messages in CosmWasm contracts

Severity: Major

In `app/app.go:783`, the CosmWasm keeper is initialized with `nil` for the `distrKeeper` parameter, with a comment stating that the `DistributionKeeper` interface is not compatible with the wasm keeper.

Any CosmWasm contract that attempts to use distribution messages (e.g., withdraw staking rewards or set a withdrawal address) will panic or fail at runtime.

The comment should be re-evaluated now that the native distribution module has been integrated, as the interface mismatch may have been resolved.

Recommendation

We recommend verifying whether the interface mismatch is still present. Other Cosmos chains have resolved this by passing `distributionkeeper.NewQuerier(app.DistributionKeeper)` rather than the keeper directly, which may satisfy the expected interface, though this should be confirmed before adoption. If a compatible adapter exists, we recommend passing it instead of `nil` and removing the stale comment.

Status: Resolved

6. Incorrect `BeginBlocker` ordering leading to inconsistent reward accounting

Severity: Major

In `app/app.go:928-935`, the `BeginBlocker` execution order invokes the slashing module before the mint and distribution modules, violating the expected execution sequence defined in the Cosmos SDK (mint -> distribution -> slashing -> staking).

This ordering contradicts both the documented assumption in the codebase and the canonical SDK design, where rewards are first minted and distributed before any slashing logic is applied. By executing slashes prematurely, slash events and validator period updates occur before reward allocation, breaking the distribution module's assumption that slashes are applied after rewards are calculated.

This misordering can lead to inconsistencies in the period-based reward accounting model, potentially resulting in incorrect reward distribution to delegators. Additionally, it may violate invariants such as CanWithdrawInvariant, which rely on the correct sequencing of reward allocation and slashing.

Recommendation

We recommend reordering the `BeginBlocker` execution to align with the Cosmos SDK standard sequence: ensure that the mint and distribution modules execute before the slashing module.

Status: Resolved

7. Identical `minDeposit` and `expeditedMinDeposit` remove economic distinctions between normal and expedited proposals

Severity: Major

In `upgrades.go:103-104`, the `setDefaultParams` function constructs governance parameters using `govtypesv1.NewParams`. The first argument sets `minDeposit` to `1,000,000,000,000 untrn`, and the second argument sets `expeditedMinDeposit` to the same value.

This is incorrect because the `x/gov` module's `ValidateBasic` function requires that `expeditedMinDeposit` is strictly greater than `minDeposit`. During the upgrade, this validation is bypassed because `Params.Set` writes directly to the storage.

Consequently, setting both deposits to the same amount removes the economic distinction between normal and expedited proposals. Expedited proposals are meant to carry a higher deposit cost to compensate for their shorter voting period (3 days vs 14 days). With identical

deposits, proposers can use the expedited path at no additional cost, undermining the governance design that trades higher financial commitment for faster resolution.

Recommendation

We recommend setting `expeditedMinDeposit` to a value greater than `minDeposit`. For example, the SDK default uses a 5x multiplier.

Status: Resolved

8. Incomplete fund migration leading to permanent token lock in deprecated module accounts

Severity: Minor

In `app/upgrades/nextupgrade/upgrades.go:410-437`, the `BurnFunds` migration function only sweeps and burns funds from the revenue-treasury module account and a hardcoded staking rewards contract address, while failing to handle other legacy module accounts associated with the deprecated revenue and feeburner modules.

Specifically, the `revenue-fee-redistribute` and `revenue-staking-rewards` module accounts are not checked or processed during the upgrade.

If any non-zero balance exists in these accounts, due to prior logic errors, failed executions, misconfigurations, or direct token transfers, those funds will remain stranded after the upgrade.

Since the corresponding modules and their storage are removed, no remaining module retains the permissions or logic required to access or transfer these funds.

This creates a permanent loss-of-funds scenario where tokens remain locked at the account level but are irrecoverable through standard protocol mechanisms.

Recommendation

We recommend extending the migration logic to explicitly query and handle all legacy module accounts, including `revenue-fee-redistribute` and `revenue-staking-rewards`.

Status: Resolved

9. 100% `minInitialDepositRatio` creates high entry barriers and renders deposit periods meaningless

Severity: Minor

In `upgrades.go:112`, the `minInitialDepositRatio` governance parameter is set to `math.LegacyOneDec()`, which is 100%. This means the proposer must deposit the full

`minDeposit` (1,000,000,000,000 untrn, approximately $5,808 at current prices) when creating the proposal. This is problematic because the standard governance flow allows proposers to submit a proposal with a partial deposit, and then the community can contribute additional funds during the deposit period to meet the threshold.

Consequently, setting the ratio to 100% renders the deposit period useless, as the full amount must be provided upfront. This creates a high financial barrier to creating proposals and removes the community's ability to signal support by co-funding them.

Recommendation

We recommend lowering `minInitialDepositRatio` to a level that enables community participation during the deposit phase.

Alternatively, if full upfront deposits are intended, consider documenting this as a deliberate design decision and shortening the deposit period accordingly.

Status: Acknowledged

10. Zero `proposalCancelRatio` enables costless governance spam

Severity: Minor

In `upgrades.go:113`, the `proposalCancelRatio` governance parameter is set to `math.LegacyZeroDec()`, which is 0%. This is problematic because the parameter controls the fraction of the deposit burned when a proposer cancels their own proposal. With a 0% cancellation ratio, a proposer can create a proposal, have it appear on governance dashboards to attract community attention, then cancel it without losing any of their deposit.

Consequently, this enables low-cost signaling attacks: an actor could repeatedly submit misleading or controversial proposals to influence market sentiment or governance discussions, cancelling them before a vote concludes. The actor does not lose anything as the deposit is fully refunded each time.

Recommendation

We recommend setting `proposalCancelRatio` to a non-zero value (e.g., the SDK default of 50%) to impose a cost on proposal cancellation.

Status: Acknowledged

11. `PuppeteerAdmin` retains migration permission post-upgrade

Severity: Minor

In `upgrades.go:272-287`, the `MigratePuppeteer` function migrates the Puppeteer contract to a new proxy contract code. The migration message uses `PuppeteerAdmin` as the sender, which is the Puppeteer contract's migration admin.

However, after the migration completes, `PuppeteerAdmin` remains the migration admin. This means `PuppeteerAdmin` can call `MigrateContract` again at any time after the upgrade, migrating the Puppeteer contract to arbitrary code.

Recommendation

We recommend clearing the contract admin after migration by calling `MsgClearAdmin`.

Alternatively, if continued admin access is intentional, consider documenting this as a design decision.

Status: Resolved

12. Truncated redelegation amounts diverge from internal accounting, leaving new validators underfunded

Severity: Minor

In `upgrades.go:544`, `calcRedelegations` constructs `MsgBeginRedelegate` messages with `Amount: sdk.NewCoin(denom, take.TruncateInt())`, which truncates the fractional `take` value to an integer.

However, in line `547`, the internal `Redelegated` tracker is updated with the untruncated value: `redelegations[newValIdx].Redelegated = redelegations[newValIdx].Redelegated.Add(take)`. This creates a growing discrepancy between what the algorithm thinks it has delegated and what the SDK actually moves.

For example, if `take` is 33.33 across three messages, `Redelegated` accumulates to 99.99 while only 99 tokens (33 + 33 + 33) are actually redelegated. The `needed` calculation in line 527 uses the inflated `Redelegated` sum, causing the algorithm to consider a validator "full" before it has actually received its target amount.

Consequently, new validators end up underfunded by the cumulative truncation error.

Recommendation

We recommend using the truncated integer value consistently for both the message amount and the internal `Redelegated` tracker. This ensures the algorithm's accounting matches the actual tokens moved by the SDK.

Status: Resolved

13. Zero-amount redelegation messages from `TruncateInt` halt the upgrade

Severity: Minor

In `upgrades.go:544`, `take.TruncateInt()` is used as the coin amount in `MsgBeginRedelegate`. The `take` value is computed as `min(remaining, needed)` in lines `531-536`, where both `remaining` and `needed` are values derived from fractional share arithmetic.

The issue is that if `take` is less than 1 (e.g., 0.5 shares remaining after prior iterations), `TruncateInt` evaluates it to 0. The SDK's `BeginRedelegate` message server checks `!msg.Amount.Amount.IsPositive()` before any further processing, rejecting the message with an `ErrInvalidRequest` error. Since `RedelegateDaoFunds` does not handle this error gracefully, the upgrade handler aborts.

Recommendation

Consider converting shares to integer token amounts before the redistribution loop to eliminate fractional arithmetic entirely.

Status: Resolved

14. `setDefaultParams` name misrepresents scope

Severity: Informational

In `upgrades.go:100-196`, the `setDefaultParams` function is documented as setting "default parameters for gov, mint, and distribution modules". However, in practice, it executes the entire upgrade logic beyond parameter configuration, such as initializing distribution state from staking, changing IBC rate-limit roles, and configuring the fee market.

Recommendation

We recommend renaming the function to accurately reflect its scope, such as `executeUpgradeSteps`.

Status: Resolved

15. `BurnFunds` error prints module name instead of address

Severity: Informational

In `upgrades.go:413`, the error message for a zero-balance revenue treasury uses the `%s` string placeholder for `RevenueModuleAccount`, which is the module name constant `"revenue-treasury"`.

This produces an error like "*revenue treasury revenue-treasury balance is not expected to be zero*", which provides no actionable information for debugging.

Recommendation

We recommend changing the error format to include the derived module address: `authtypes.NewModuleAddress(RevenueModuleAccount)`.

Status: Resolved

16. `removeSchedule` error references removed admin module

Severity: Informational

In `wasmbinding/message_plugin.go:861`, the error message for unauthorized `removeSchedule` calls reads "*only admin or security dao can remove schedule*".

However, the admin module has been removed as part of the upgrade, so only the security DAO address is checked in line `860`. The error message is misleading because it suggests admin module authorization still exists.

Recommendation

We recommend updating the error message to "*only the security dao can remove schedule*" to accurately reflect the current authorization logic.

Status: Resolved

17. Stale comment incorrectly states that `GovMsg::VoteWeighted` does not work with Neutron

Severity: Informational

In `app/app.go:793-794`, a comment states that the `cosmwasm_1_2` capability enables `GovMsg::VoteWeighted`, which does not work with Neutron because it uses its own custom governance. Since this PR migrates Neutron to the native Cosmos SDK `gov` module, `GovMsg::VoteWeighted` may now work as expected and the comment no longer reflects the current state of the codebase.

Recommendation

We recommend verifying whether `GovMsg::VoteWeighted` works correctly with the new governance setup and, if so, removing or updating the comment accordingly.

Status: Resolved

18. Multiple hardcoded values in the upgrade handler will cause deployment failure

Severity: Informational

In `app/upgrades/nextupgrade/upgrades.go:51-69`, several constants required by the upgrade handler are left as placeholder values:

- `NewMaxValidators = 1`
- `PuppeteerAdmin = "TODO"` (an invalid address used as the sender in `MigratePuppeteer`)
- `ProxyContractCodeID = 0x0000` (resolves to `0`, a non-existent code ID used in the same migration)
- `UndelegationsManagerContract = "TODO"` (an invalid address used as the cron schedule target in `RegisterCronSchedules`)
- `NewValidatorSet` contains a single placeholder validator address with a TODO comment indicating it has not been finalized

If deployed in this state, the upgrade handler will fail during execution. `MigratePuppeteer` will error on the invalid sender address or missing code ID, and `RegisterCronSchedules` will register a cron pointing at an invalid contract.

Also, hard-coding the staking module's `MaxValidators` parameter to 1 forces the Tendermint consensus set to a single bonded validator and collapses the chain's security model from Byzantine fault tolerance to complete trust in one operator.

Recommendation

We recommend filling in all values with the correct production addresses and code IDs before deploying this upgrade, and adding a preflight validation at the start of `CreateUpgradeHandler` that asserts all constants are valid bech32 addresses and that `ProxyContractCodeID` is non-zero.

Status: Acknowledged

19. `SetupStaking` writes staking params twice via two different code paths

Severity: Informational

In `app/upgrades/nextupgrade/upgrades.go:447-459`, `SetupStaking` first writes the updated `MaxValidators` directly to the store via `sk.SetParams`, then immediately re-writes the same params through `stakingMsgServer.UpdateParams`.

The two calls are redundant: if `SetParams` succeeds, `UpdateParams` will also succeed and overwrite the same data.

Recommendation

We recommend removing the direct `sk.SetParams` call and relying solely on `stakingMsgServer.UpdateParams`, or vice versa, to avoid the redundant double write.

Status: Resolved

20. `BurnFunds` fails hard if either account balance is zero at upgrade time

Severity: Informational

In `app/upgrades/nextupgrade/upgrades.go:412-422`, `BurnFunds` returns an error if the revenue treasury or the staking rewards contract holds a zero balance, causing the entire upgrade handler to abort. If either account is empty at the upgrade block, the upgrade cannot complete. This makes the function sensitive to on-chain state at the time of upgrade scheduling.

Recommendation

We recommend replacing the hard failures with log warnings when a zero balance is encountered, allowing the upgrade to proceed gracefully. Alternatively, the expected balances should be verified in a dry-run against a mainnet state snapshot before the upgrade is scheduled.

Status: Resolved

21. Transitive redelegation rule may block `RedelegateDaoFunds` if recent redelegations exist

Severity: Informational

In `app/upgrades/nextupgrade/upgrades.go:486`, `RedelegateDaoFunds` calls `BeginRedelegate` for each Puppeteer delegation. The Cosmos SDK enforces a transitive redelegation prevention rule: if a delegator has an incoming redelegation to a given source validator that is still within the unbonding period (20 days on Neutron), a subsequent `BeginRedelegate` from that validator will fail with `ErrTransitiveRedelegation`, aborting the upgrade.

We classify this issue as informational, as the Drop puppeteer contract has been verified on-chain to have no active redelegations at the time of this review.

Recommendation

We recommend verifying before scheduling the upgrade that no redelegations to any current Puppeteer source validator have occurred within the preceding 20 days. Consider adding a runtime check at the start of `RedelegateDaoFunds` that scans for active incoming redelegations and skips or logs affected validators rather than failing hard.

Status: Acknowledged